

Speaker-Independent Verbatim Transcription of Overlapping Meeting Speech with Near-Real-Time Streaming on Consumer Hardware

Shreya Mary Kurien, Krishiv Saluja, Sampath Bageyawadi, Thummala Koushik, M Sanjay
Department of Computer Science and Engineering
Woxsen University, Hyderabad, India

Abstract—Transcribing a conversation recorded by one microphone in a room requires deciding who is speaking, recovering overlapped speech, keeping disfluencies such as “um” that off-the-shelf recognisers delete, and doing so fast enough to be useful live. We present an open-weight pipeline that combines neural diarisation, overlap-aware routing, two-source separation with voice-based stream-to-speaker assignment, and verbatim Whisper recognition, plus a streaming variant with stable speaker labels. On an Apple M3 laptop, an Apple-GPU recogniser and call packing bound streaming latency at about 7–8s, down from a growing 14.5s mean. We then adapt Whisper-small to verbatim far-field meeting speech on the same 16GB laptop by training its top encoder layers and decoder from cached activations. A first version, trained on windows transcribing every speaker, kept fillers but raised word error rate (WER), because inside the pipeline the recogniser only ever hears one speaker. Building training windows from single-speaker stretches with the pipeline’s own routing removes this mismatch: on 300 held-out AMI utterances, verbatim WER falls from 37.7% to 32.2%, WER from 33.9% to 31.5%, and filler recall rises from 2.3% to 64.8%. In full-pipeline scoring of three four-person meetings the adapted model keeps 65 of 83 fillers instead of 2, with cpWER unchanged within its confidence intervals; diarisation, not recognition, dominates the remaining error.

Index Terms—speaker diarization, speech separation, overlapped speech, verbatim ASR, disfluency, streaming, Whisper, AMI

I. INTRODUCTION

Meeting transcription from a shared room microphone is difficult for reasons that compound. Speakers talk over each other; the recogniser must attribute words to the right person; and downstream users of a transcript (minute takers, researchers studying conversation, accessibility tools) often need the speech as it was said, including fillers, repetitions and pauses. Large weakly supervised recognisers such as Whisper [1] are strong general-purpose transcribers but are trained to produce clean text: on far-field meeting speech they routinely delete fillers and backchannels. Conversely, systems built for overlapped speech typically target a fixed evaluation setup and are not designed to run live on a personal computer.

This paper describes an end-to-end pipeline, and reports what we measured while building it, with three aims: (i) speaker independence, meaning it works for an arbitrary set of speakers with no per-speaker training and only optional enrolment for real names; (ii) verbatim output; and (iii) near-

real-time streaming on consumer hardware. Our contributions are:

- A modular pipeline in which overlap flags and speech boundaries are derived from a single diarisation pass, overlapped segments are separated by a two-source model, and separated streams are mapped back to diarised speakers by voice similarity, which is necessary because separator output order is arbitrary (Sec. III).
- A streaming variant with a session-long speaker registry that stabilises `Speaker_N` labels using temporal continuity between overlapping windows and voice similarity only as a fallback (Sec. IV).
- Engineering measurements that determined whether the system runs live: an Apple-GPU Whisper backend, a cost model showing Whisper call time is independent of clip length, and packing of clean clips into shared 30s windows, including a case where packing hurt accuracy (Sec. VI-B).
- A memory-lean recipe for adapting Whisper-small to verbatim far-field meeting speech on a 16GB laptop, the diagnosis of why a first version hurt accuracy (a train/inference speaker mismatch), and a fix that keeps fillers *and* improves WER (Sec. V).

II. RELATED WORK

Diarisation. pyannote.audio [2] combines a frame-level segmentation network, trained with a powerset loss that models simultaneous speakers [3], with embedding clustering. We use its output turns, which keep overlaps, instead of separate voice-activity and overlap detectors. Online variants such as Diart [4] keep a persistent bank of speaker centroids to label a stream incrementally.

Separation. Deep clustering [5] and permutation-invariant training [6] made single-channel multi-talker separation practical; SepFormer [7] is a transformer separator trained on synthetic mixtures such as Libri2Mix [8] or the noisy, reverberant WHAMR! [9]. Output order is arbitrary per segment, so a continuous pipeline must re-associate streams with speakers; continuous speech separation is benchmarked on LibriCSS [10].

Verbatim and streaming recognition. Whisper [1] is a strong general transcriber but is trained toward clean text and

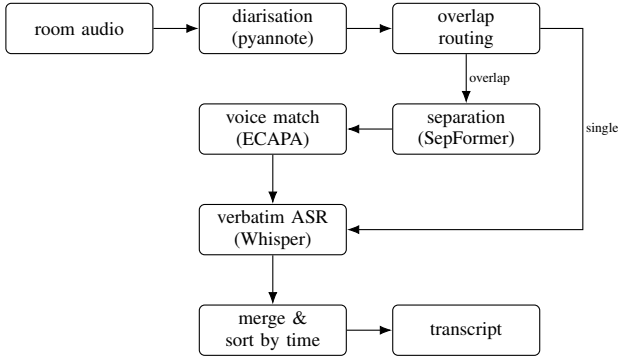


Fig. 1. Pipeline. Single-speaker segments go straight to recognition; overlapped segments are separated, each stream is matched to a diarised speaker by voice, then recognised. Lines are merged in time order.

tends to delete disfluencies [11]. CrisperWhisper [12] fine-tunes Whisper for verbatim output and precise word timing. For live use, `whisper_streaming` [13] commits text once consecutive hypotheses agree; our system instead commits audio older than a fixed guard, which suits re-diarising a window.

Meeting evaluation. The AMI corpus [14] provides scenario meetings with headset and distant-microphone audio and verbatim transcripts; we use its single-distant-microphone (SDM) condition, the closest public match to a laptop on a table. Recent distant-meeting benchmarks include CHiME-6 [15] and NOTSOFAR-1 [16]. CHiME popularised concatenated minimum-permutation WER (cpWER), which penalises wrong words and words credited to the wrong speaker. We report it with diarisation error rate (DER, `pyannote.metrics` [17]), cross-check it with MeetEval [18], which also provides the time-constrained tcpWER, and give bootstrap intervals [19].

III. SYSTEM

Fig. 1 shows the pipeline. All models are open-weight and run locally.

A. Diarisation and overlap routing

We use the `pyannote speaker-diarization-3.1` pipeline. Its turns keep simultaneous speakers as overlapping turns, so one pass yields both speech/non-speech boundaries and overlap flags. We cut the timeline at every turn boundary, label each piece with its set of active speakers, and merge neighbouring pieces with identical sets separated by at most 0.4 s. Overlaps shorter than 0.4 s are folded into the neighbouring speaker because SepFormer is unreliable on such short clips and they are typically backchannels or boundary jitter. The result is a list of segments (t_s, t_e, S) , treated as overlapping when $|S| \geq 2$.

B. Separation and stream-to-speaker assignment

Overlapped segments are separated by a SepFormer checkpoint at 8 kHz into two streams (resampling is handled internally). Both outputs are scaled by one shared factor so their relative levels are preserved; a stream whose RMS is below

10% of the louder one is treated as leakage and skipped, since recognisers hallucinate words on such residue.

Because the output order is arbitrary, we compute for each diarised speaker a voice print from their *clean* (non-overlapped) speech using ECAPA-TDNN [20] embeddings (SpeechBrain [21]), embed each separated stream, and solve a one-to-one assignment maximising total cosine similarity with the Hungarian algorithm [22]. Speakers who never talk alone have no clean print; they are paired positionally with any unmatched stream as a last resort. Optional enrolment reuses the same embedder: an enrolled clip’s embedding is matched to speaker prints above a threshold, replacing the generic label by a name.

C. Verbatim recognition and boundary handling

Whisper’s default decoding suppresses a set of non-speech tokens, which we disable. Each segment is transcribed with 0.4 s of extra context on both sides, and only words whose timestamp midpoint lies inside the segment are kept, so a word straddling a boundary is assigned to exactly one segment and none is chopped. For separated streams the crop is applied at an edge only when a neighbouring segment also transcribes the same speaker; otherwise a speaker who *starts* inside the overlap would lose a first word whose timestamp Whisper stretches back into leading silence. Gaps between words of at least 0.6 s are rendered as explicit pause markers. Decoding is greedy with a temperature fallback (0, 0.2, 0.4) when the output is repetitive (compression ratio above 2.4) or low-confidence; generation is capped at 8 tokens per second of audio plus 24, and a segment still above compression ratio 2.8 is dropped. On separated streams, utterances consisting only of fillers are discarded, because the fine-tuned model emits confident lone “mm” tokens on separation residue.

IV. STREAMING

The streaming variant reuses every stage on a rolling buffer. Every ≥ 4 s of new audio, the last 16 s are re-diarised; only audio older than a 3 s guard is *committed*, so emitted lines are never retracted. The latency of a line is therefore roughly the guard plus processing time plus the wait for the next step. Steps run in a worker thread; a step never starts while another is running, so on slow hardware the hop grows instead of the system falling behind.

Stable speaker labels. Each window’s diarisation produces arbitrary local labels. We map them to session-long identities with a registry of running voice prints, using two kinds of evidence in order of trust: (1) *temporal continuity*: consecutive windows share most of their audio, so a local speaker active at the same moments as a previously labelled speaker inherits its label (Hungarian matching on overlapped seconds, minimum 0.4 s); (2) *voice similarity* against registered prints (cosine threshold 0.4) for speakers with no temporal anchor. Two safeguards mattered in practice. A print computed only from overlapped speech is marked *weak*: it may create a provisional identity but never updates an existing speaker’s print. A speaker present only inside the guard region is not registered

at all until they appear in committed audio; without this, a phantom duplicate identity was created for a speaker whose first second fell in the unstable tail. A speaker-count hint is applied as an upper bound, since forcing an exact count on a short window makes the diarizer split one voice into two.

V. VERBATIM ADAPTATION ON AMI

A. Data

We use the utterance-level AMI SDM release (CC BY 4.0), whose transcripts are verbatim (fillers such as *um*, *uh*, *hmm*, *mm-hmm* are kept, roughly 1 in 40 words). Utterances average 3 s, but Whisper always encodes a padded 30 s window, so we rebuild each meeting’s timeline by placing clips at their start times (all clips of a meeting come from the same microphone) and cut it into windows of up to 28 s, labelled in Whisper’s native format with timestamp tokens so segment and word timing keep working. Text is lower-cased without punctuation (AMI’s convention); each window is loudness-normalised (RMS 0.05), as at inference.

Single-speaker windows. Our first version windowed *all* speakers’ utterances together, so a training target was everyone’s words in time order. The pipeline’s recogniser never sees that: it receives either a clean single-speaker segment or one separated stream (Sec. III). The mismatch taught the model to transcribe whoever was audible. The current version runs the pipeline’s own overlap-routing function over each meeting’s reference turns and windows only utterances that lie inside single-speaker stretches, so every training pair is “this audio, this one speaker’s words”. Overlapped stretches are excluded, and windows containing a pathological word-repeat run are dropped. We use 14 of 27 training shards, chosen evenly across the split, giving 17 704 windows (21.7 h, 78 meetings). Windows average only 4.4 s, because single-speaker stretches in meetings are short; this matches the short segments the pipeline transcribes; two validation shards and three test shards are held out, with no meeting shared between splits.

B. Cached-encoder fine-tuning

On the Apple M3 (MPS), for a batch of four 30 s windows the Whisper-small encoder costs 2.81 s forward+backward versus 0.46 s for the decoder, so full fine-tuning would take about 4.5 h per 1 200 steps. We instead run the frozen lower ten encoder layers *once* and cache the hidden states entering layer 10 as float16 (8.3 GB for training), then train the top two encoder layers, the final encoder norm and the whole decoder (168 M of 242 M parameters) from the cache. The decoder learns the verbatim style and meeting vocabulary; the top encoder layers adapt to far-field acoustics. Training uses AdamW [23] (learning rate 10^{-5} , weight decay 0.01) with linear warm-up and decay, effective batch 8 (2 windows $\times$ 4 accumulation; a micro-batch of 4 exhausted the 16 GB machine), and 1 400 optimiser steps. Because the cache holds hidden states rather than features, augmentation masks the cached activations directly (two 100-frame time spans and one 64-channel band per window, training set only), in the spirit of SpecAugment [24] applied in latent space as in

TABLE I
SEPARATION QUALITY (SI-SDR [28], DB; SI-SDR_i IS IMPROVEMENT OVER THE UNPROCESSED MIXTURE). ONE MIXTURE PER ROW.

Mixture	Checkpoint	SI-SDR s_1/s_2	Mix	SI-SDR _i
TTS+TTS	Libri2Mix	19.9 / 18.4	0.2	19.0
TTS+TTS	WSJ0-2mix	8.9 / 6.5	0.2	7.6
TTS+TTS	WHAMR!	1.4 / 1.8	0.2	1.5
Real+TTS	Libri2Mix	13.4 / 11.6	0.1	12.4

wav2vec 2.0 [25]. Every 200 steps we greedy-decode 30 validation windows and keep the checkpoint with the lowest validation WER (fillers scored), rather than the last one; validation WER was lowest at the first evaluation, step 200 (19.6%), and did not improve over the remaining 1 200 steps (1 400 steps are 0.63 epochs), so the reported model is the step-200 checkpoint. The gain therefore comes from a short adaptation; later steps did not help, and a shorter schedule or lower learning rate is the obvious next experiment. Optimiser and schedule state are saved with every checkpoint, so an interrupted run resumes exactly; the run was started fresh, not from the earlier model. The fine-tuned weights are converted to MLX format; our converter reproduces the published stock MLX weights exactly (479 tensors, maximum difference 0) before being used on the fine-tuned model.

VI. EXPERIMENTS

Setup. All timing and training is on one fanless Apple M3 laptop with 16 GB unified memory. Software: PyTorch 2.14, pyannote.audio 4.0.7, SpeechBrain 1.1.1, faster-whisper [26] 1.2.1 (CTranslate2), MLX [27] Whisper, transformers 5.17. WER uses Whisper’s English normaliser (lower-casing, spelling and number normalisation, fillers removed) on both sides; *verbatim WER* keeps fillers; *filler recall* is the fraction of reference fillers (multiset) present in the output. DER uses a 0.25 s collar with overlap scored. Brackets give 95% percentile-bootstrap intervals (1 000 resamples) of the ratio of summed errors to summed reference words, over utterances, or over 30 s blocks under a fixed speaker mapping for a single meeting’s cpWER.

A. Separation and overlap detection

Table I compares three public SepFormer checkpoints on a two-voice conversation with a 2.5 s overlap, generated with two macOS text-to-speech voices and mixed digitally with known ground truth, and on a 6.5 s overlap between a real recorded voice and a TTS voice. The checkpoint trained on clean Libri2Mix [8] mixtures is far better than those trained on WSJ0-2mix [5] or the noisy, reverberant WHAMR! [9]; we caution that both mixtures are clean digital sums, which favours clean-trained models, and that each row is a *single* mixture. Overlap detection on the first mixture placed the overlap at 5.01–7.56 s against a true 5.0–7.5 s.

On the TTS+TTS conversation, the full pipeline reached 0.0% DER and 0.0% cpWER with separation, against 45.1% cpWER with the same diariser but without separation (the

TABLE II
WHISPER TIME FOR A 4.4 S CLIP (GREEDY DECODING, WORD
TIMESTAMPS). RTF = TIME / AUDIO DURATION.

Model	CTranslate2, CPU		MLX, Apple GPU	
	s	RTF	s	RTF
small	2.34	0.53	0.45	0.10
base	0.72	0.16	0.21	0.05
tiny	0.38	0.09	0.13	0.03

TABLE III
STREAMING LATENCY (SECONDS FROM SPEECH TO EMITTED LINE) ON A
21 S TWO-SPEAKER CLIP WITH OVERLAP REPLAYED AT REAL-TIME PACE.
SINGLE RUN PER ROW; RUN-TO-RUN VARIATION IS VISIBLE BETWEEN THE
LAST TWO ROWS.

Configuration	Mean	Max
CPU ASR (CTranslate2), CPU diarisation	14.5	25.3
MLX ASR + MPS torch models	7.4	10.1
+ pack all clips (loses overlap words)	5.9	7.6
+ pack clean clips only (final)	8.2	11.3

overlapped stretch collapsed into a single mixed-speaker line); this is a single, easy example and is not evidence of general performance.

B. Speed

Backend. Table II shows Whisper transcription time for a 4.4 s clip. The Apple-GPU (MLX) backend is 5.2× faster than CTranslate2 on CPU for Whisper-small with an identical transcript. Smaller models are faster still but visibly worse on overlapped audio (e.g. tiny produced “Sometimes my Gwen’s day...”), so we kept small. Beam size and thread count had no measurable effect on the CPU path.

Call cost is independent of clip length. Whisper encodes a padded 30 s window, so on MLX a 1 s clip took 0.77 s and an 8 s clip 0.89 s. Cost therefore scales with the *number* of calls, and a streaming step containing several segments pays for each. Packing clips into one window separated by 1 s of silence and splitting words back by timestamp cuts the call count to about one per step. Packing *everything*, however, measurably damaged overlaps: in one overlapped window a whole utterance (“this is a test for our DL pipelines”) disappeared. Packing only clean single-speaker clips and transcribing separated streams alone restored the transcript exactly (Table III).

In the first row latency grew during overlaps (each step cost more than the audio it covered); in the others it stays bounded. Latency is dominated by the 3 s guard, the wait for the next step, and diarisation of the 16 s window (1–2.6 s on MPS). This is “a few seconds” only loosely; we do not claim sub-second operation.

C. Verbatim adaptation

Table IV reports 300 held-out utterances (16 test meetings, 17.7 min), the same utterances for both models. Stock Whisper-small keeps 3 of 128 reference fillers even with token

TABLE IV
AMI SDM HELD-OUT UTTERANCES (300, 16 MEETINGS, GREEDY
DECODING, BOTH MODELS WITH THE REPETITION GUARD OF SEC. III).
BRACKETS: 95% BOOTSTRAP INTERVALS OVER UTTERANCES.

Model	WER	Verbatim WER	Filler recall
Stock	33.9 [30.3, 38.3]	37.7 [34.3, 41.8]	2.3 [0.0, 5.3]
Adapted	31.5 [27.6, 35.9]	32.2 [28.2, 36.4]	64.8 [53.4, 74.8]

suppression disabled. The adapted model keeps 83 (64.8%) and is also more accurate: WER 31.5% against 33.9%, and 32.2% against 37.7% when fillers are scored as words.

What the first version got wrong. The earlier model, trained on all-speaker windows and evaluated on a different 300-utterance sample, had kept 74% of fillers but raised WER from 38.8% to 42.5%. Its error decomposition showed why: deletions fell from 15.3% to 8.0% of reference words, as expected from verbatim training, but insertions rose from 10.4% to 28.8%. An SDM utterance clip contains other participants’ speech while its reference contains only the labelled speaker, and the model, trained to transcribe everyone audible, did exactly that. A second cause was repetition loops on short clips (“a f making a f making ...”), handled at inference by a temperature fallback and a generation cap of 8 tokens per second of audio plus 24. Removing the speaker mismatch in the training data is what turned the WER change from +3.7 to −2.4 points.

The verbatim-WER and filler-recall gains are large relative to their 95% intervals. For plain WER the two per-model intervals overlap, so this sample alone does not establish the 2.4-point gain; since both models are scored on the same utterances, a paired bootstrap is the appropriate test, and our evaluation scripts provide it. The robust claims are therefore that the adapted model is no less accurate than stock Whisper and substantially more verbatim.

D. Meeting-level results

Table V scores the whole pipeline on 5-minute excerpts of three AMI test meetings with four speakers each. Overlap ranges from 5% of speech (IS1009b) to 41% (EN2002b: 106 s, of which 24 s have three or more simultaneous talkers, beyond the two-source separator by design). DER uses the complete speaker-turn annotation. The text reference of ES2004c lacks 78 s of speech that the turn annotation contains; an earlier version of our evaluation built the DER reference from text turns and reported a spurious 57% DER, and cpWER on that excerpt still counts correctly recognised words in those 78 s as insertions.

At meeting level the clear effect is disfluency retention: 65 of 83 reference fillers (78%) against 2 (2%). Word accuracy changes by +3.2, −2.2 and +1.3 cpWER points, well inside intervals that span tens of points; three 5-minute excerpts cannot distinguish the recognisers, and the utterance-level comparison above is the informative one. The time-constrained tcpWER is somewhat higher for the adapted model on two

TABLE V
FULL-PIPELINE SCORES ON 5-MINUTE AMI SDM EXCERPTS (4 SPEAKERS). CPWER AND TCPWER (5 S COLLAR) IGNORE FILLERS; BRACKETS: 95% BLOCK-BOOTSTRAP INTERVALS. DER IS IDENTICAL FOR BOTH RECOGNISERS (SAME DIARISER).

Meeting	DER	cpWER		tcpWER	Fillers kept
		Stock	Adapted	St./Ad.	St. → Ad.
ES2004c	9.4	58.7 [49, 85]	61.9 [47, 89]	62.9/67.1	1/14 → 14/14
IS1009b	25.4	56.0 [32, 90]	53.8 [24, 91]	58.4/56.0	1/39 → 30/39
EN2002b	23.5	55.0 [51, 63]	56.3 [54, 71]	57.1/62.6	0/30 → 21/30
Mean / total	19.4	56.6	57.3	59.5/61.9	2/83 → 65/83

excerpts, which suggests its timestamps are less precise; we have not yet analysed this.

The absolute error rates are high for both recognisers and should not be compared with published AMI numbers, which use different segmentation and reference handling and often headset audio. The DER breakdown shows that the dominant errors are upstream of recognition: IS1009b has 21.2% speaker confusion (two of its speakers are labelled consistently only 52–53% of the time), EN2002b misses 15.4% of speech, and every diarisation error propagates to cpWER.

VII. DISCUSSION AND LIMITATIONS

The training target must match what the recogniser will hear. The same model, data source, layers and budget went from hurting accuracy to helping it once each training window contained one speaker, as the pipeline guarantees at inference. For components that sit inside a pipeline, the inference-time routing is part of the data specification; reusing the routing code to build the training set made this match exact rather than approximate.

Punctuation. Because AMI transcripts carry none, the fine-tuned model emits lower-case unpunctuated text, a readability regression relative to stock Whisper. A punctuation-restoration pass would address this; we have not implemented it.

Scale and statistical strength. The separation and end-to-end synthetic results are single mixtures, often generated with TTS voices; latency figures are single runs on one machine; the utterance test uses 300 utterances; meeting-level scoring covers three 5-minute excerpts, whose intervals are wide; we trained one configuration on 14 of 27 shards without a learning-rate sweep, and its best checkpoint came at the first validation point; and validation WER for checkpoint selection uses 30 windows. The results should be read as a systems study with indicative measurements, not a benchmark.

Design limits. Separation is two-source; three or more simultaneous speakers cannot all be recovered. Overlap quality is bounded by diarisation: if the diariser misses a second speaker, their words are attributed to the first (observed once in our tests). Speakers who never speak alone have no clean voice print and fall back to positional pairing. The default separator was chosen on clean audio and should be re-evaluated on noisy rooms, where the WHAMR!-trained checkpoint may behave differently. The browser front end was verified only

with replayed recordings and unit tests for its resampling logic, not with a live microphone.

Future work. Diarisation is now the largest error source, so a stronger or adapted diariser is the next lever; a separated-stream training set would match the recogniser’s other input path; more evaluation meetings (and corpora such as NOTSOFAR-1 or CHiME-6) are needed for meeting-level conclusions; diarisation-conditioned recognition would remove the two-speaker separation limit; and incremental streaming diarisation would lower the latency floor set by re-diarising a window.

VIII. CONCLUSION

We built a speaker-independent pipeline that separates overlapped speech, attributes it to consistent speakers, keeps disfluencies, and streams on a laptop with bounded latency of roughly 7–8 s. The largest speed gains came from measuring cost structure (GPU Whisper, per-call rather than per-second cost, label stabilisation) rather than from model changes. For verbatim recognition, matching the training targets to what the recogniser hears inside the pipeline was decisive: the adapted Whisper-small keeps about two thirds of fillers that the stock model deletes while lowering verbatim WER from 37.7% to 32.2% on held-out utterances without any loss in standard WER. At meeting level the remaining errors are dominated by diarisation, which is where we see the most room for improvement.

DEMONSTRATION AND REPRODUCIBILITY

A browser demo (`./run.sh demo`) plays bundled recordings, including the AMI excerpts, into the live pipeline at real-time pace while playing the same audio, shows speaker-coloured lines with overlap tags and highlighted fillers beside the ground-truth transcript, and switches between the stock and fine-tuned recognisers.

All code, the 88-test unit suite, data-preparation, training (`training/run_all.sh`, resumable), conversion and evaluation scripts are in the project repository; AMI is available under CC BY 4.0. Training and evaluation used fixed seeds; per-utterance hypotheses and meeting transcripts are saved by the evaluation scripts.

REFERENCES

- [1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.
- [2] H. Bredin, “pyannote.audio 2.1 speaker diarization pipeline: Principle, benchmark, and recipe,” in *Proc. Interspeech*, 2023.
- [3] A. Plaquet and H. Bredin, “Powerset multi-class cross entropy loss for neural speaker diarization,” in *Proc. Interspeech*, 2023.
- [4] J. M. Coria, H. Bredin, S. Ghannay, and S. Rosset, “Overlap-aware low-latency online speaker diarization based on end-to-end local segmentation,” in *Proc. IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2021.
- [5] J. R. Hershey, Z. Chen, J. Le Roux, and S. Watanabe, “Deep clustering: Discriminative embeddings for segmentation and separation,” in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2016.

- [6] M. Kolbæk, D. Yu, Z.-H. Tan, and J. Jensen, "Multitalker speech separation with utterance-level permutation invariant training of deep recurrent neural networks," *IEEE/ACM Trans. Audio, Speech, and Language Processing*, vol. 25, no. 10, 2017.
- [7] C. Subakan, M. Ravanelli, S. Cornell, M. Bronzi, and J. Zhong, "Attention is all you need in speech separation," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2021.
- [8] J. Cosentino, M. Pariente, S. Cornell, A. Deleforge, and E. Vincent, "LibriMix: An open-source dataset for generalizable speech separation," arXiv:2005.11262, 2020.
- [9] M. Maciejewski, G. Wichern, E. McQuinn, and J. Le Roux, "WHAMR!: Noisy and reverberant single-channel speech separation," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2020.
- [10] Z. Chen, T. Yoshioka, L. Lu, T. Zhou, Z. Meng, Y. Luo, J. Wu, X. Xiao, and J. Li, "Continuous speech separation: Dataset and analysis," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2020.
- [11] E. E. Shriberg, "Preliminaries to a theory of speech disfluencies," Ph.D. dissertation, University of California, Berkeley, 1994.
- [12] M. Zusag, L. Wagner, and B. Thallinger, "CrisperWhisper: Accurate timestamps on verbatim speech transcriptions," in *Proc. Interspeech*, 2024.
- [13] D. Macháček, R. Dabre, and O. Bojar, "Turning Whisper into real-time transcription system," in *Proc. IJCNLP-AACL: System Demonstrations*, 2023.
- [14] J. Carletta *et al.*, "The AMI meeting corpus: A pre-announcement," in *Machine Learning for Multimodal Interaction (MLMI)*, ser. Lecture Notes in Computer Science, vol. 3869. Springer, 2005.
- [15] S. Watanabe *et al.*, "CHiME-6 challenge: Tackling multispeaker speech recognition for unsegmented recordings," in *Proc. 6th Int. Workshop on Speech Processing in Everyday Environments (CHiME)*, 2020.
- [16] A. Vinnikov *et al.*, "NOTSOFAR-1 challenge: New datasets, baseline, and tasks for distant meeting transcription," in *Proc. Interspeech*, 2024.
- [17] H. Bredin, "pyannote.metrics: A toolkit for reproducible evaluation, diagnostic, and error analysis of speaker diarization systems," in *Proc. Interspeech*, 2017.
- [18] T. von Neumann, C. Boeddeker, M. Delcroix, and R. Haeb-Umbach, "MeetEval: A toolkit for computation of word error rates for meeting transcription systems," in *Proc. 7th Int. Workshop on Speech Processing in Everyday Environments (CHiME)*, 2023.
- [19] M. Bisani and H. Ney, "Bootstrap estimates for confidence intervals in ASR performance evaluation," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2004.
- [20] B. Desplanques, J. Thienpondt, and K. Demuynck, "ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification," in *Proc. Interspeech*, 2020.
- [21] M. Ravanelli *et al.*, "SpeechBrain: A general-purpose speech toolkit," arXiv:2106.04624, 2021.
- [22] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [23] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.
- [24] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Proc. Interspeech*, 2019.
- [25] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [26] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," <https://github.com/SYSTRAN/faster-whisper>, 2023.
- [27] A. Hannun, J. Digani, A. Katharopoulos, and R. Collobert, "MLX: Efficient and flexible machine learning on Apple silicon," <https://github.com/ml-explore/mlx>, 2023.
- [28] J. Le Roux, S. Wisdom, H. Erdogan, and J. R. Hershey, "SDR – half-baked or well done?" in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2019.